

FRACTIONAL NUMBERS

float, double, and long double

THE NEED

Not every number is whole

An amount spent needs a fractional part - cents. C++ has three floating-point types for that, each trading off size for precision.

```
float amountSpent{93.333333333333f};  
double averageDailySpend{0.185185185185};  
long double preciseAverage{93.33333333333333L};
```

Size, precision, significant digits

- **Size** - how many bytes of memory a variable is given. Think of it as the size of the box a value has to fit in.
- **Significant digits** - how many digits of a number you can actually *trust* to be correct, reading left to right. **93.333** has 5 of them; the digits after that are just filler if the box was too small to remember more.
- **Precision** - a type's *limit* on significant digits. A bigger box (more bytes) can afford to remember more digits, so it has higher precision.

A kitchen scale that only shows whole grams isn't "wrong" when you put a 93.3 g apple on it and it reads 93 g - it simply wasn't built to be more precise than that. A float is that scale; a double is a much more precise one.

PRECISION COMPARED

How many digits can each type trust?

TYPE	SIZE (THE BOX)	SIGNIFICANT DIGITS (WHAT YOU CAN TRUST)
<code>float</code>	4 bytes	~7
<code>double</code>	8 bytes	~15-16
<code>long double</code>	platform-dependent	more than <code>double</code>

A bigger box doesn't just hold a bigger number - it holds a *more precise* one, because there's room to remember more of its digits.

SEEING THE LIMIT FOR YOURSELF

Ask each type to show you more digits than it has

```
std::println("amountSpent      : {:.15f}", amountSpent);  
std::println("averageDailySpend: {:.15f}", averageDailySpend);
```

`{:.15f}` asks for 15 digits after the decimal point - more than a `float` can actually trust. Past its ~7 significant digits, what prints is just leftover noise, not the real value: this is precision running out, made visible.

WHY IT MATTERS FOR MONEY

Digit drift

```
float totalSpentThisYear = 1'924'000.23f; // looks fine printed normally...  
std::println("{:.5f}", totalSpentThisYear); // ...but isn't, at this precision
```

float's limited precision is why it's a poor fit for a running total of money - this "obviously wrong" digit drift quietly corrupts a ledger over time if left unnoticed.

SCIENTIFIC NOTATION

For very large or very small numbers

```
double roundingFeeFraction{3.498e-11}; // 3.498 * 10^-11
double annualRevenue{1.924e8};         // 1.924 * 10^8
```

Handy once numbers get very large (annual revenue) or very small (the fraction of a cent a currency-conversion fee might round to).

DIVIDING BY ZERO

Infinity and NaN

```
double totalSpent{-5.6};  
double expenseCount{};           // zero expenses logged yet  
  
double infinityResult{totalSpent / expenseCount}; // -inf  
double nanResult{0.0 / expenseCount};           // nan
```

Floating-point division by zero doesn't crash the way integer division does - it produces special values instead: infinity, or "not a number" when the result is undefined (like `0.0 / 0.0`).

DEBUG IT

Inspect the special values directly

Breakpoint after `infinityResult` and `nanResult` are computed - your IDE's locals view will show `inf`/`-inf` and `nan` directly, which is the clearest way to recognize them the first time you see one in the wild.